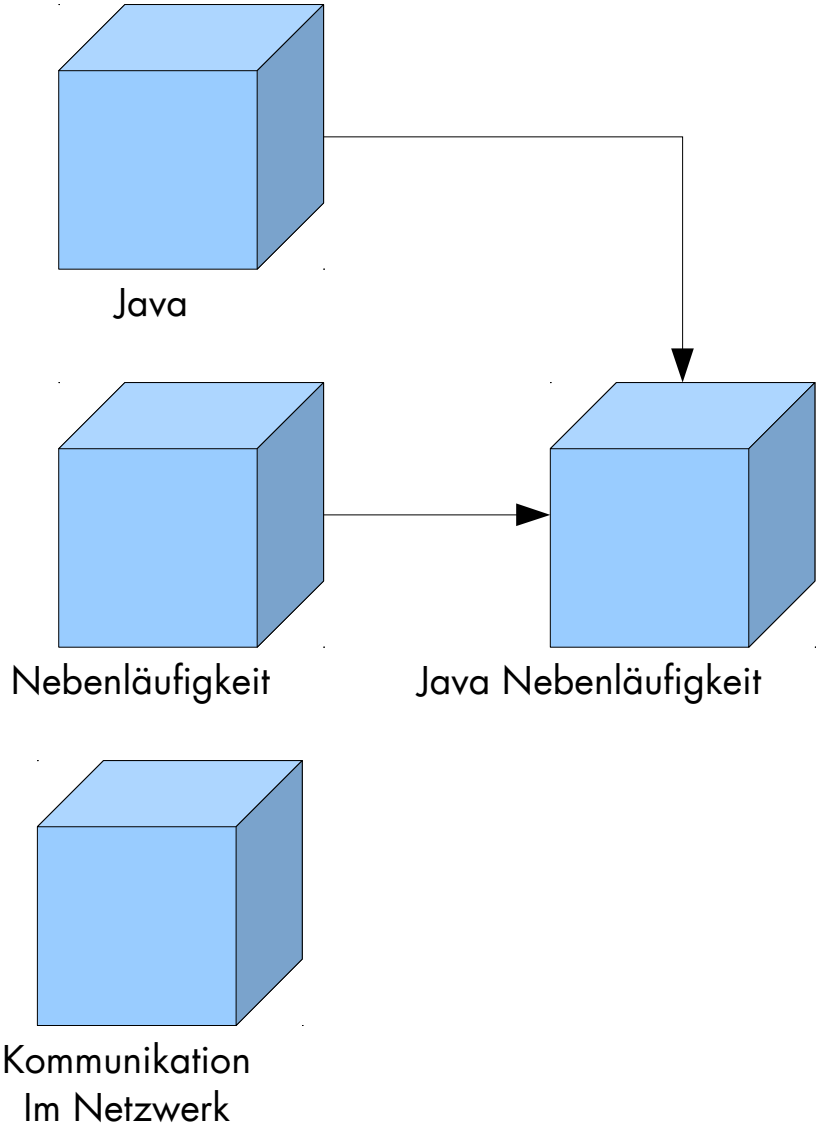
UNIVERSITÄT LEIPZIG
Institut für Wirtschaftsinformatik

Entwicklung verteilter Anwendungen

Interprozesskommunikation

Pascal Kovacs

5. Mai 2020

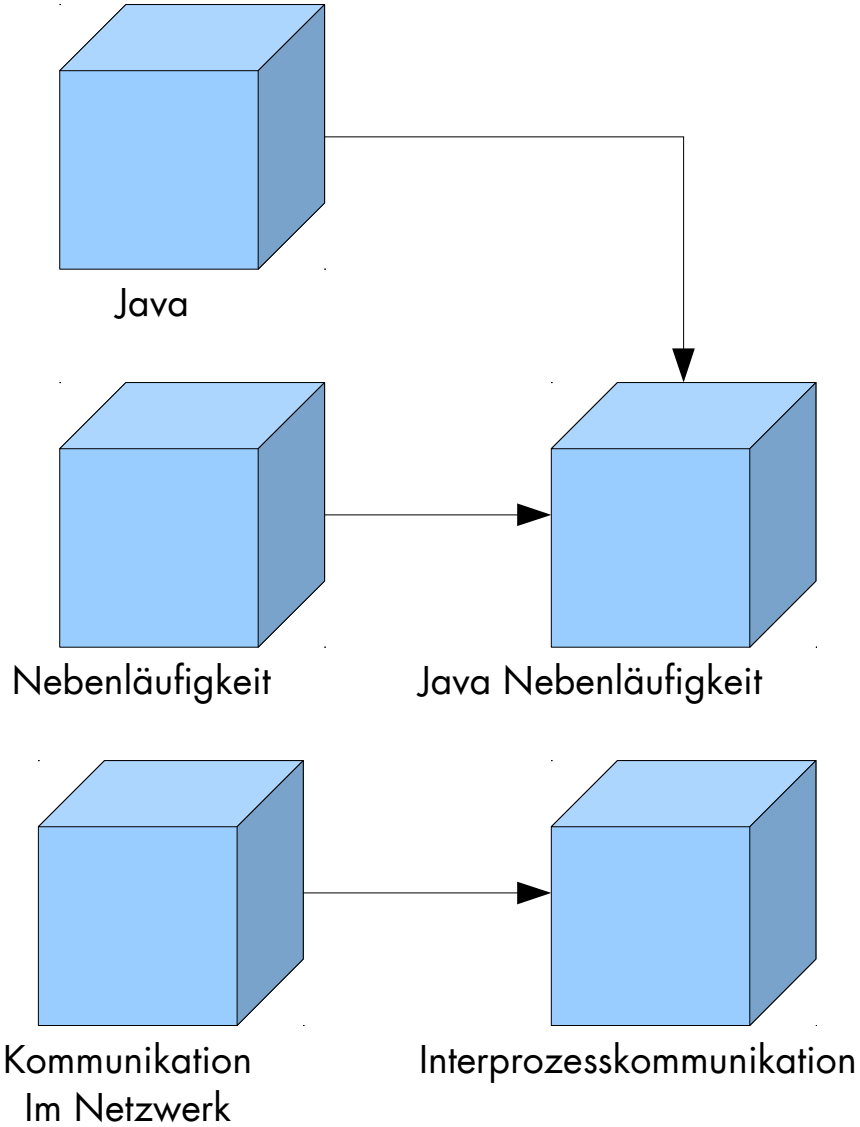

Java
Nebenläufigkeit
Java Nebenläufigkeit
Kommunikation
Im Netzwerk

Ziele Netzwerkkommunikation

- Netztopologien mit Vor- und Nachteilen kennen
- Logische Netzwerkmodelle verstehen
- ISO/OSI Referenzmodell Aufbau und Funktionsweise
- 3 Dimensionen der Anfragen kennen

Kontrollfragen Netzwerkkommunikation

- Welche Netztopologien gibt es?
- Welche Vor- und Nachteile haben sie?
- Wie ist das ISO/OSI Referenzmodell aufgebaut?
- Was ist die Aufgabe der Sicherungsschicht?
- Was ist die Aufgabe der Transportschicht?
- Welche Dimensionen für Abfragen gibt es?
- Was ist der "zentrale" Unterschied zwischen den beiden logischen Netzwerkmodellen?


Java
Nebenläufigkeit
Java Nebenläufigkeit
Kommunikation
Im Netzwerk
Interprozesskommunikation

Ziele Interprozesskommunikation

- IPK und Methoden verstehen
- IP und Ports
- TCP
- UDP
- Theorie Sockets

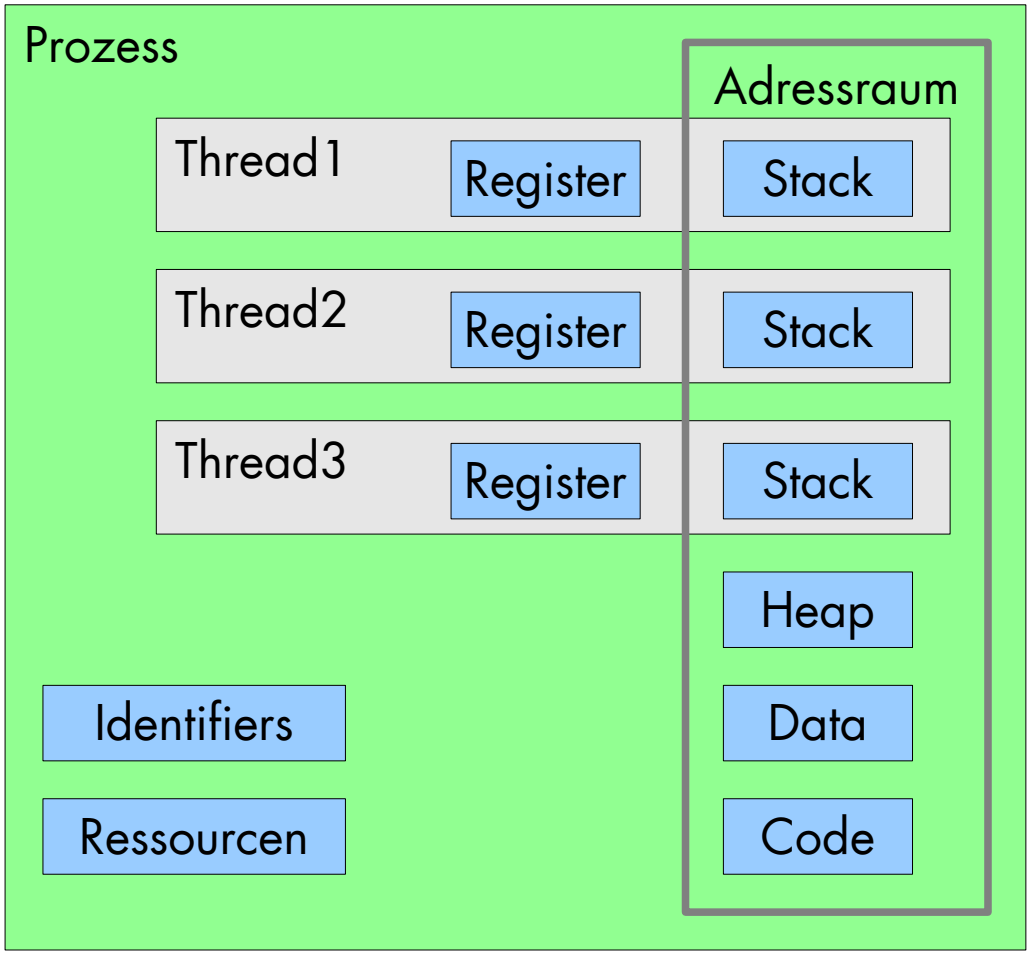
Kontrollfragen Interprozesskommunikation

- Welche Methoden zur IPK gibt es?
- Wie hängen Netzwerkkommunikation und Interprozesskommunikation zusammen?
- Was bestimmen IP-Adressen?
- Was bestimmen Ports?
- Was ist der zentrale Unterschied zwischen TCP und UDP?
- Welche Eigenschaften ergeben sich daraus für TCP und UDP?
- Zu welchen Schichten des ISO/OSI Referenzmodells gehören IP, TCP und UDP?
- Wie viele Sockets braucht es für ein- und ausgehenden IPK von zwei Teilnehmern?
- Wie funktionieren TCP und UDP mittels Sockets?

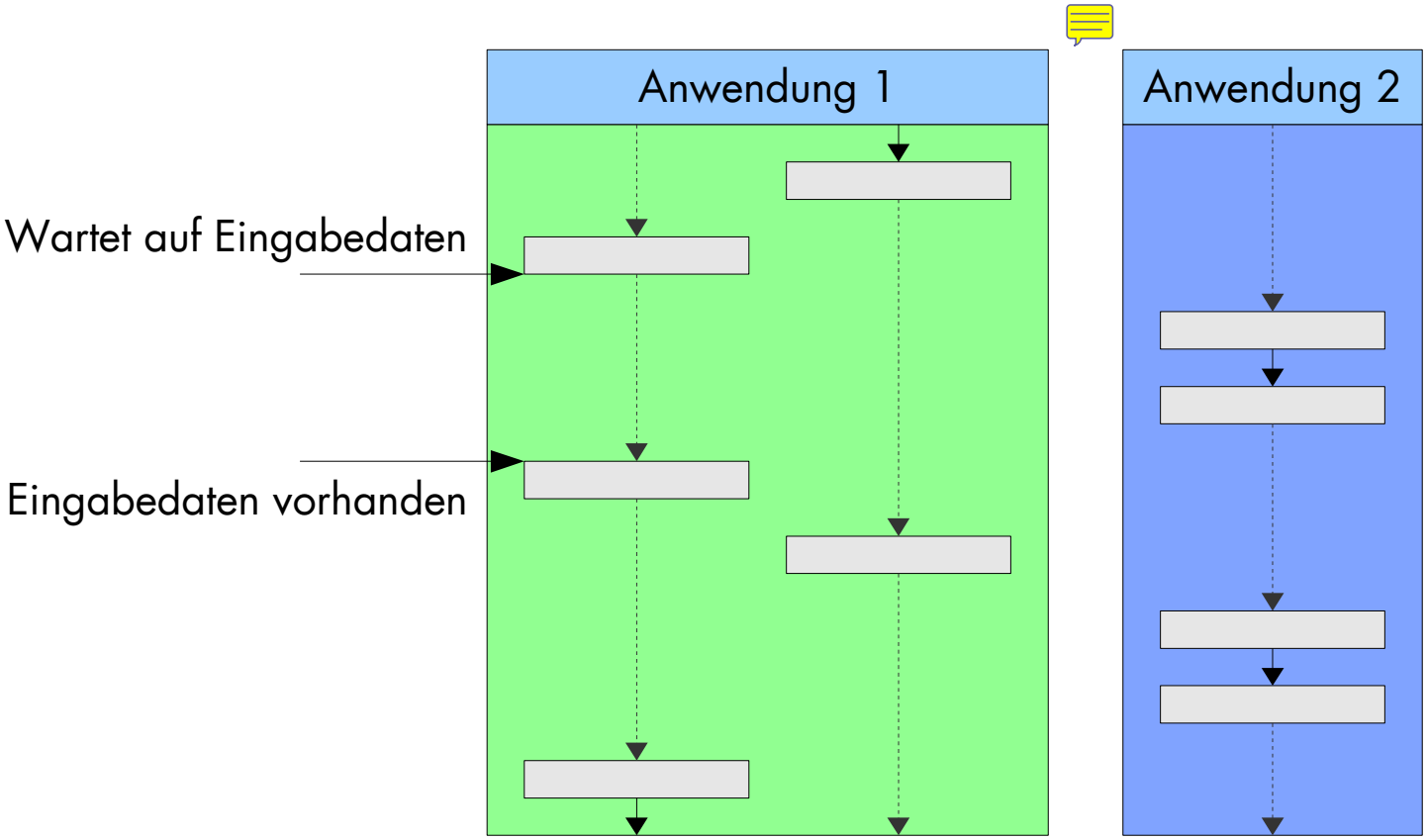
Übersicht

- Grundlagen Interprozesskommunikation
- Internet Protokoll
- Transmission Control Protocol
- User Datagram Protocol
- Sockets
- Realisierung in C++

Wiederholung Threads und Prozesse



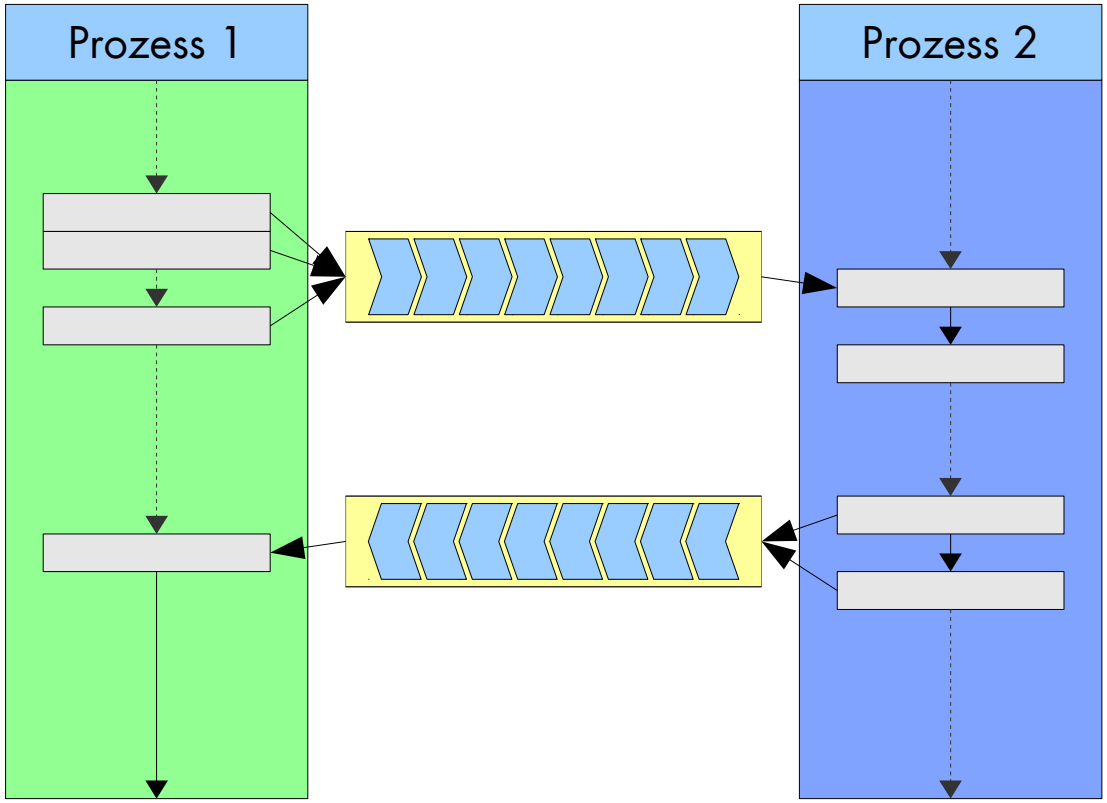
Wiederholung Threads und Prozesse



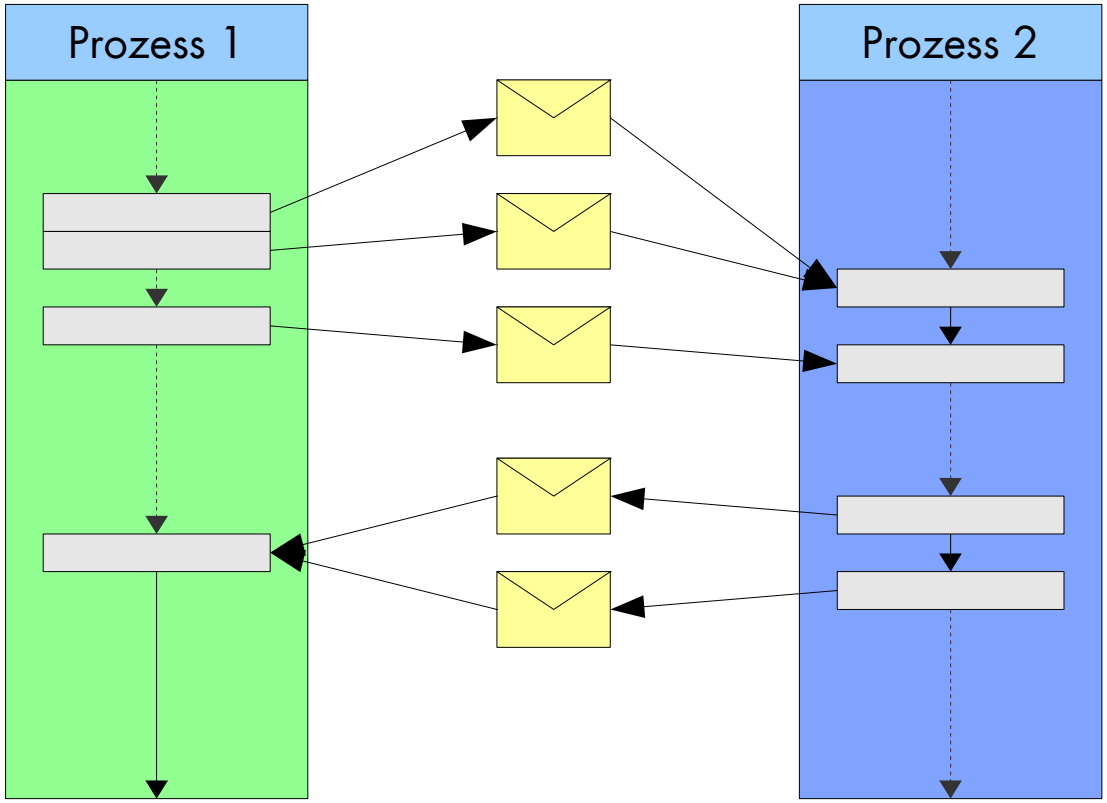
Interprozesskommunikation (IPK)

- Prozesse laufen isoliert voneinander ab
- Kein gemeinsamer Speicherbereich
- Werden vom Betriebssystem isoliert verwaltet
- Mehrere Prozesse können nebenläufig ausgeführt werden
- Koordination mehrerer Prozesse erfolgt durch IPK
- Methoden der IPK sind
  - Datenströme (Pipes)
  - Ereignisse (Events)
  - Nachrichten (Message Queues)
  - Speziell für IPK bereitgestellter Speicher/Ressourcen (Shared Memory)

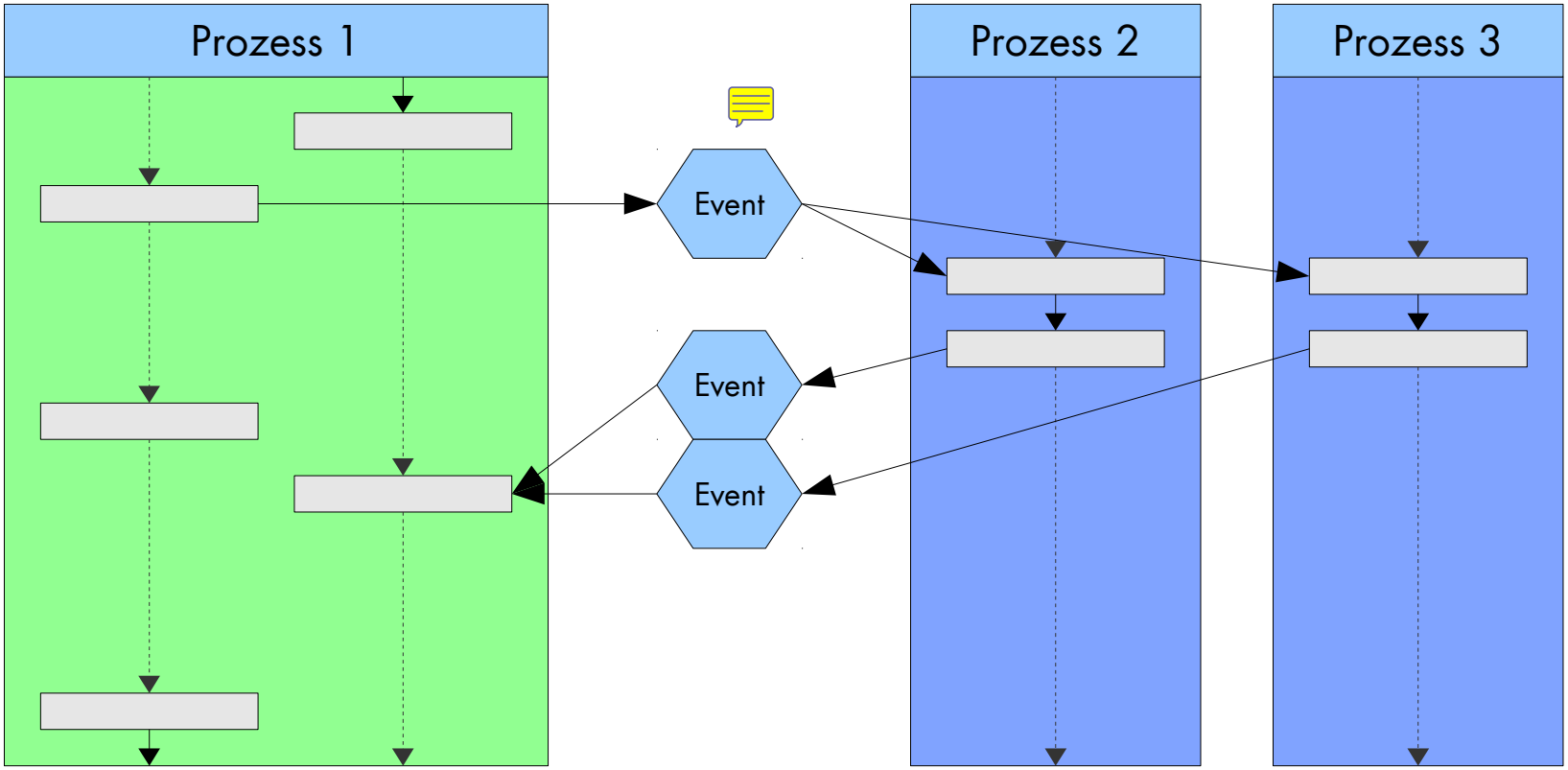
IPK Pipes



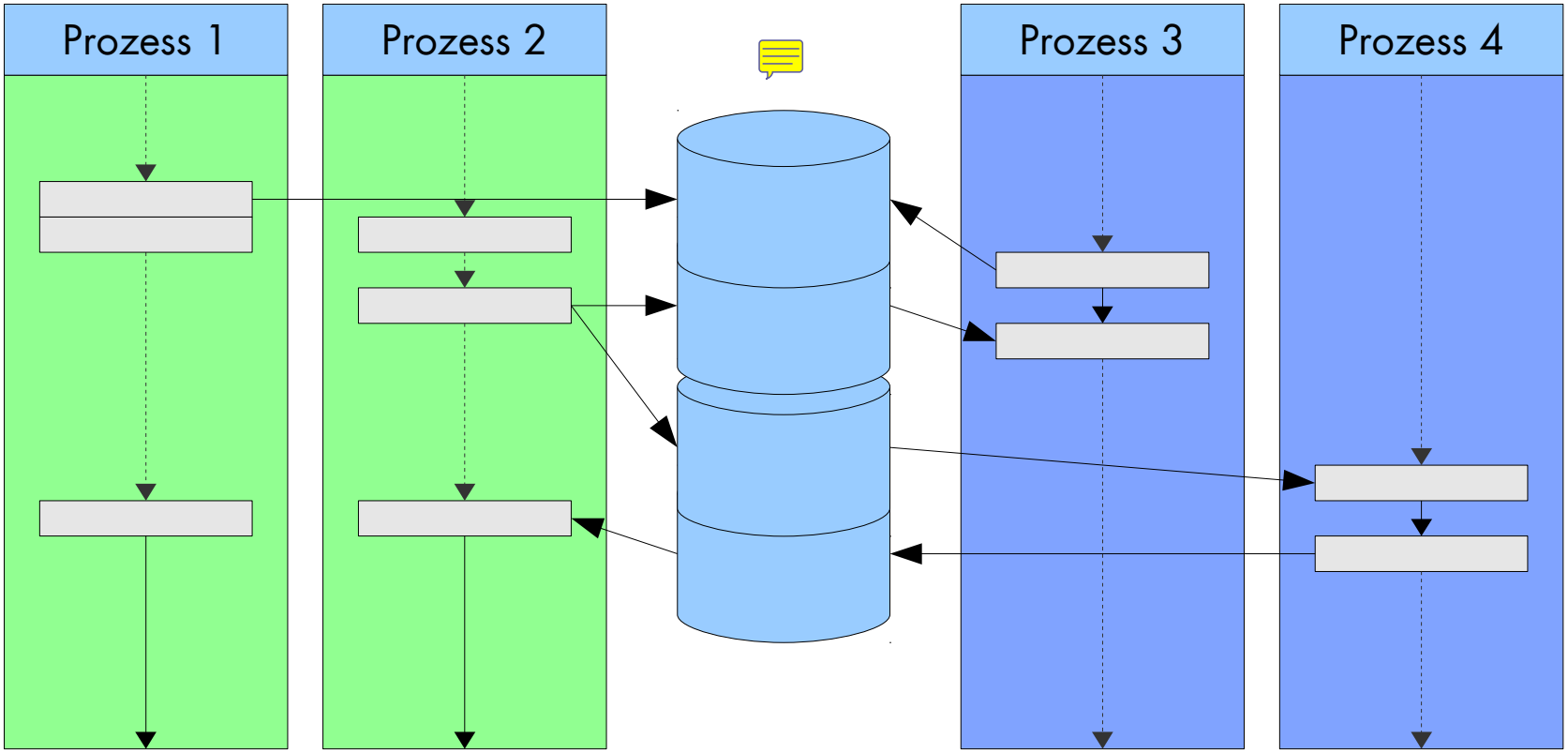
IPK Messages



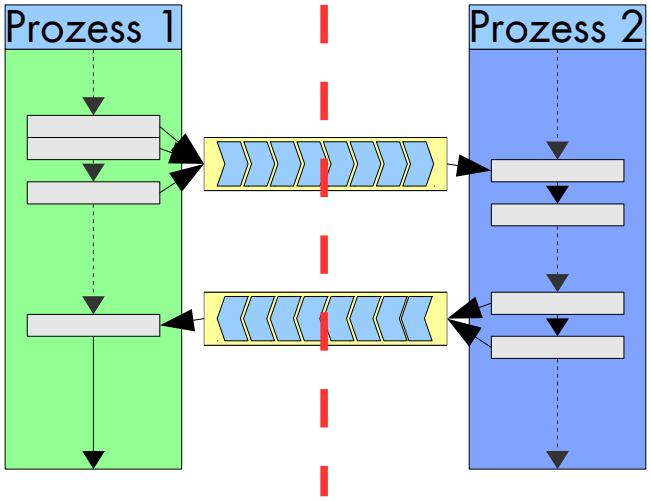
IPK Events

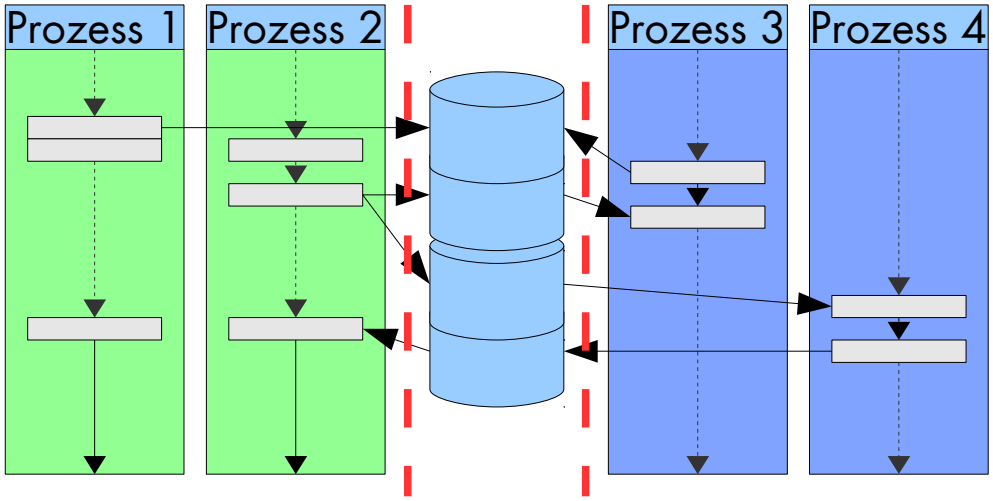


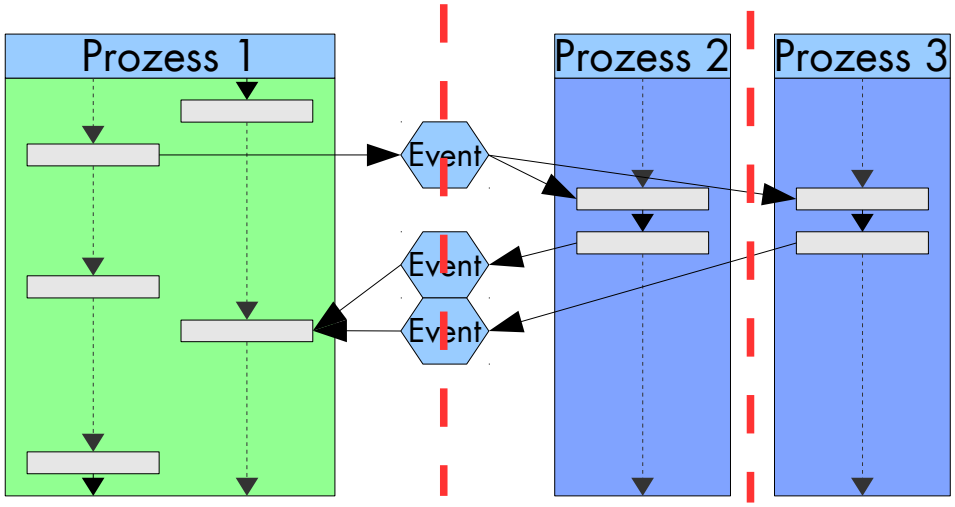
IPK shared Memory

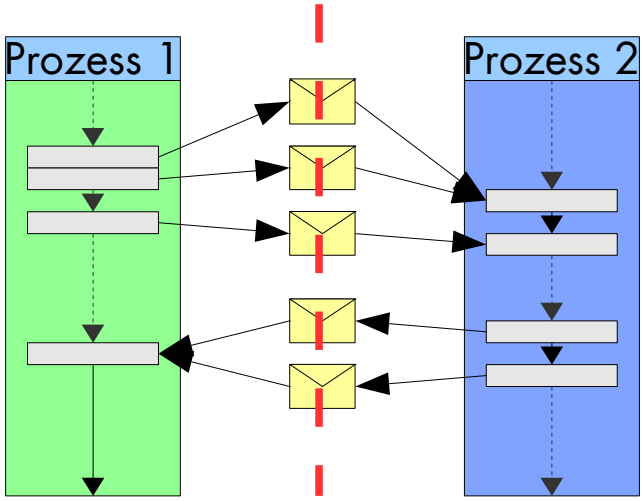


IPK über das Netzwerk



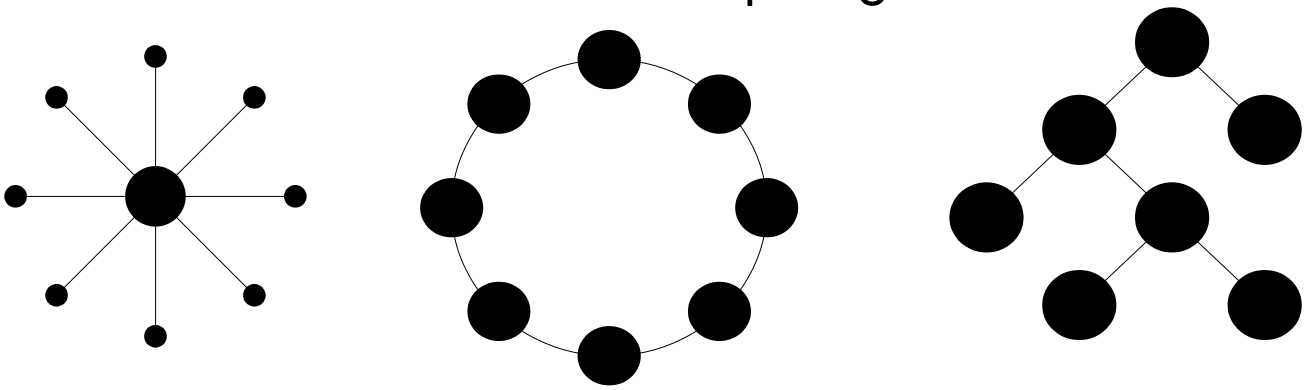




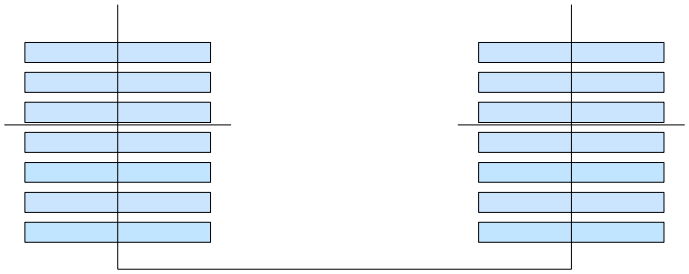


Bisher in Netzwerkkommunikation behandelt

- Aufbau von Netzwerken in Topologien

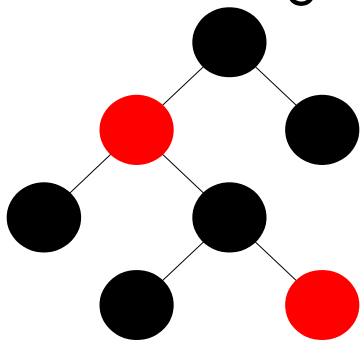


- Schichtenmodell zur Kommunikation zwischen zwei Knoten

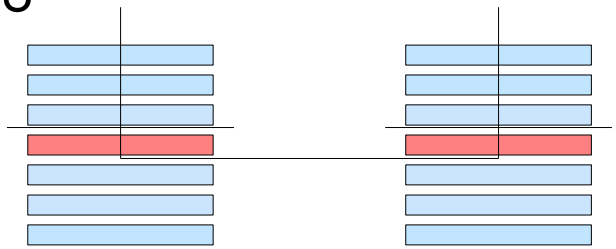


Ergänzende Inhalte zur IPK im Netzwerk

- Identifizierung der einzelnen Knoten im Netzwerk mittels Adressen



- Logische Kommunikation auf Ebene der Transportschicht

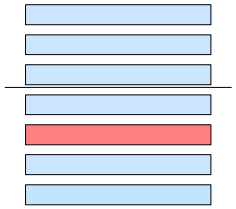


- Verbindungsorientierte vs. Verbindungslose Kommunikation
- Realisierung der Kommunikation auf Grundlage von Sockets

Übersicht

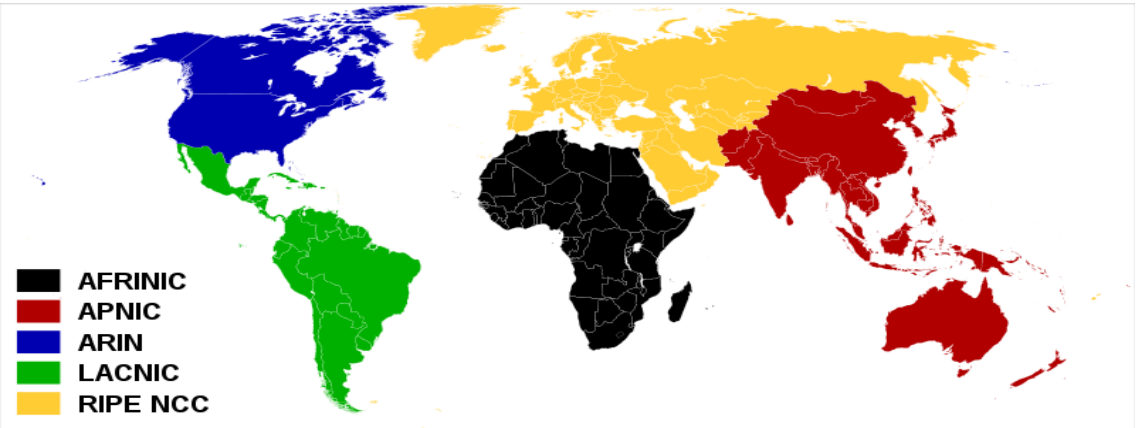
- Grundlagen Interprozesskommunikation
- Internet Protokoll
  - Grundlagen
  - IPv4
  - IPv6
- Transmission Control Protocol
- User Datagram Protocol
- Sockets
- Realisierung in C++

IP (Internet Protocol)



- Protokoll der Vermittlungsschicht
- Erste vom Übertragungsmedium unabhängige Schicht
- Gewährleistet Eindeutigkeit der IP-Adresse in einem Netzwerk
- Somit eindeutige Identifikation der Kommunikationsendpunkte
- Gruppiert Computer in einem Netzwerke in logische Einheiten
- Nutzt dafür eindeutigen Adresse und Subnetzmaske
- Somit Wegewahl und Weiterleitung von Paketen (Routing) möglich

IP (Internet Protocol)



- Zuordnung öffentlicher, weltweit eindeutiger IP-Adressen durch die Internet Assigned Numbers Authority (IANA)
- Vergabe großer Netze an Regional Internet Registries (RIR)
- RIR vergeben Subnetze an Internet-Service-Provider (ISP)
- Vergabe von IP-Adressen an Endkunden
- Weitere Subnetzbildung durch ISP möglich

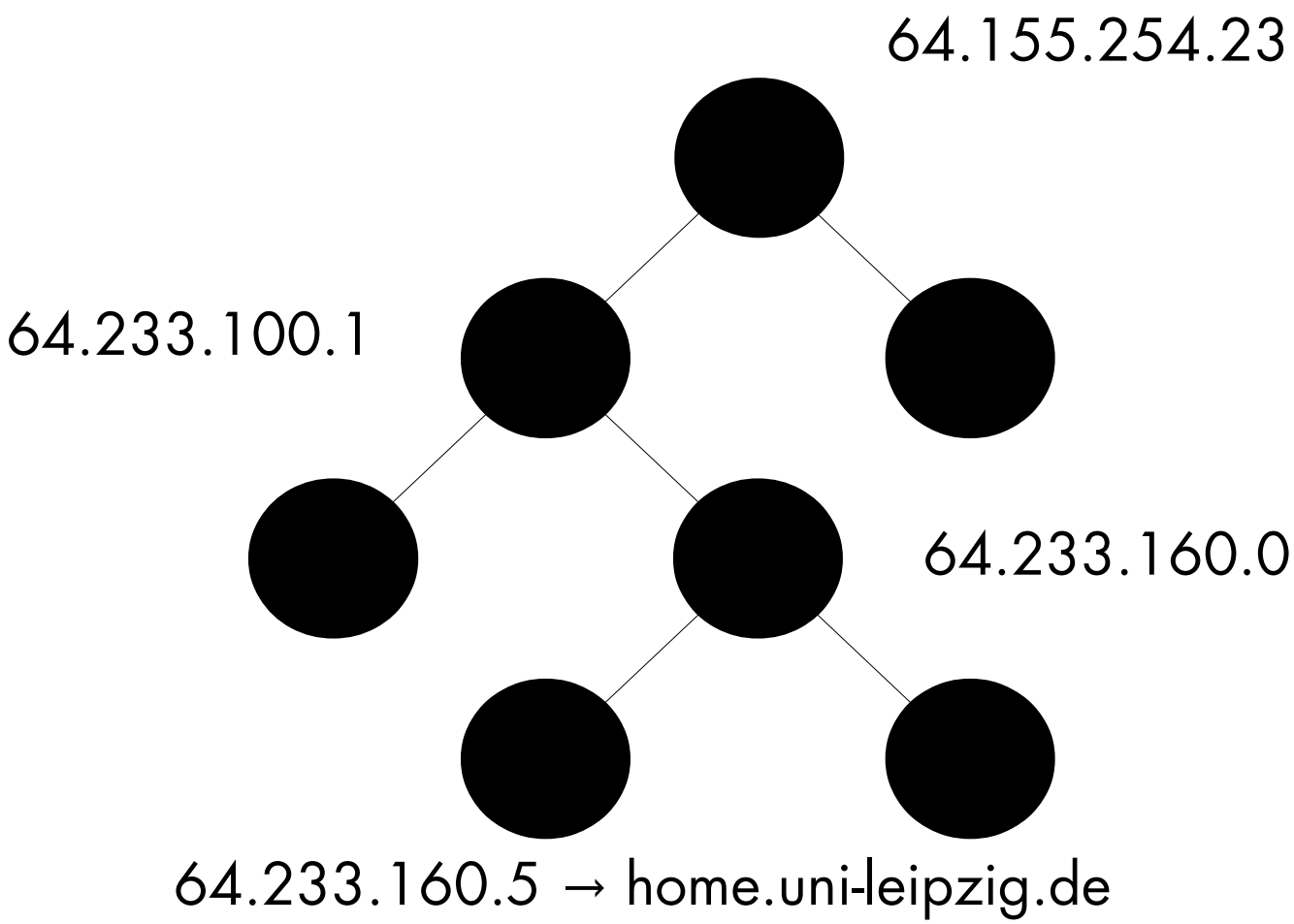
IP-Adresse

- Adresse in Computernetzen, die auf dem Internetprotokoll basieren
- Zuweisung zu Geräten registriert dieses im Netzwerk
- Datenpakete werden mit einer IP-Adresse versehen, die den Empfänger eindeutig identifiziert
- Grundlage für Routing-Entscheidungen bei Zustellung der Datenpakete
- Bsp: 192.168.42.23

Domain Name System (DNS)

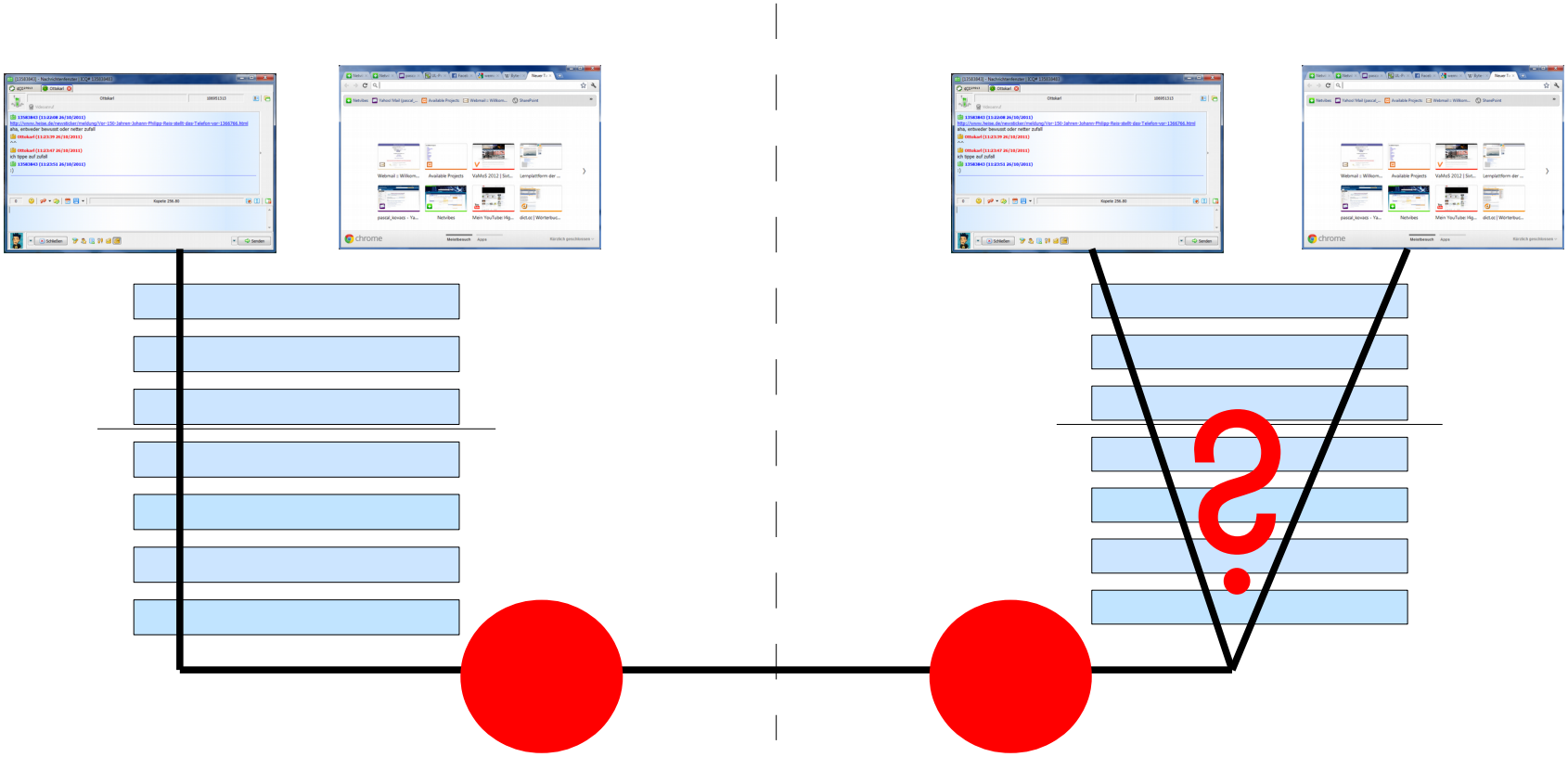
- IP-Adressen für Menschen schwer lesbar (64.233.160.0 → Google)
- Bezeichner statt Zahlen aussagekräftiger
- Zuordnung (Auflösung) von Namen zu IP-Adressen und umgekehrt
- Analog zum Prinzip der Telefonauskunft
- Zur Skalierung wird ein verteiltes, hierarchisches System (Verzeichnisdienst) genutzt
- Aufbau: Rechnernamen.Sub-Level-Domain(s).Top-Level-Domain
  - z.B. home.uni-leipzig.de
- Vordefinierter Name für 127.0.0.1: localhost
- Vergabe von Top Level Domains erfolgt durch die IANA

Identifikation der Knoten per IP-Adresse



Identifikation der Anwendung/Prozess

- IP-Adresse dient lediglich der Identifikation des Knotens
- Pro Knoten können aber mehrere Anwendungen/Prozesse laufen



Port

- Definiert zusätzlich zur IP-Adresse eine weitere Adresskomponente
- Identifikation von Prozessen auf Zielknoten möglich
- Netzwerkkommunikation mehrerer Prozesse kann aufrecht erhalten werden, ohne dass eine Verbindung unterbrochen werden muss
- Standard Zuordnung der *Ports* 0 bis 49151 zu bestimmten Diensten durch die IANA
  - http://www.iana.org/assignments/port-numbers

Aufteilung der Port-Nummern

- Portnummern (0 bis 65.535)
  - *Well known Ports*: 0 bis 1023
    - *File Transfer Protocol* (FTP): 21
    - *Telnet*: 23
    - *Secure Shell* (SSH): 22
    - *Simple Mail Transfer Protocol* (SMTP): 25
    - *Hyper Text Transfer Protocol* (HTTP): 80
  - *Registered Ports*: 1024 bis 49151
  - *Dynamic and/or private Ports*: 49152 bis 65535

Übersicht

- Grundlagen Interprozesskommunikation
- Internet Protokoll
  - Grundlagen
  - IPv4
  - IPv6
- Transmission Control Protocol
- User Datagram Protocol
- Sockets
- Realisierung in C++

IPv4 Aufbau

- Die IP-Adresse umfasst 32 Bit
  - Darstellung mittels vier Oktets (Bytes)
  - Damit sind 2^{32} = 4.294.967.296 IP-Adressen darstellbar
- Darstellung
  - Dotted decimal notation – Darstellung mittels vier punktseparierten Dezimalszahlen von 0 bis 255
  - Bsp.: 192.168.0.7 = 11000000.10101000.00000000.00000111

Netzwerkteil und Geräteteil

- IPv4-Adressen werden durch eine Netzmaske (ebenfalls 32Bit), in
  - Einen Netzwerk- und
  - Einen Geräteteil („Hostteil“) getrennt
- Netzmaske gibt an, an welchem Bit die Adresse geteilt wird
- Von der Netzmaske maskierte Bits (Netzwerkteil) sind bei allen Hosts (Rechnern) eines Subnetzwerks identisch
- Wird von Host benötigt, um Routing-Entscheidungen zu treffen
- Netzwerkteil und Geräteteil werden durch die Netzmaske und bitweisem AND Verknüpfung ermittelt

IPv4 Beispiel

Beipiel: IP-Adresse 139.94.122.195/27

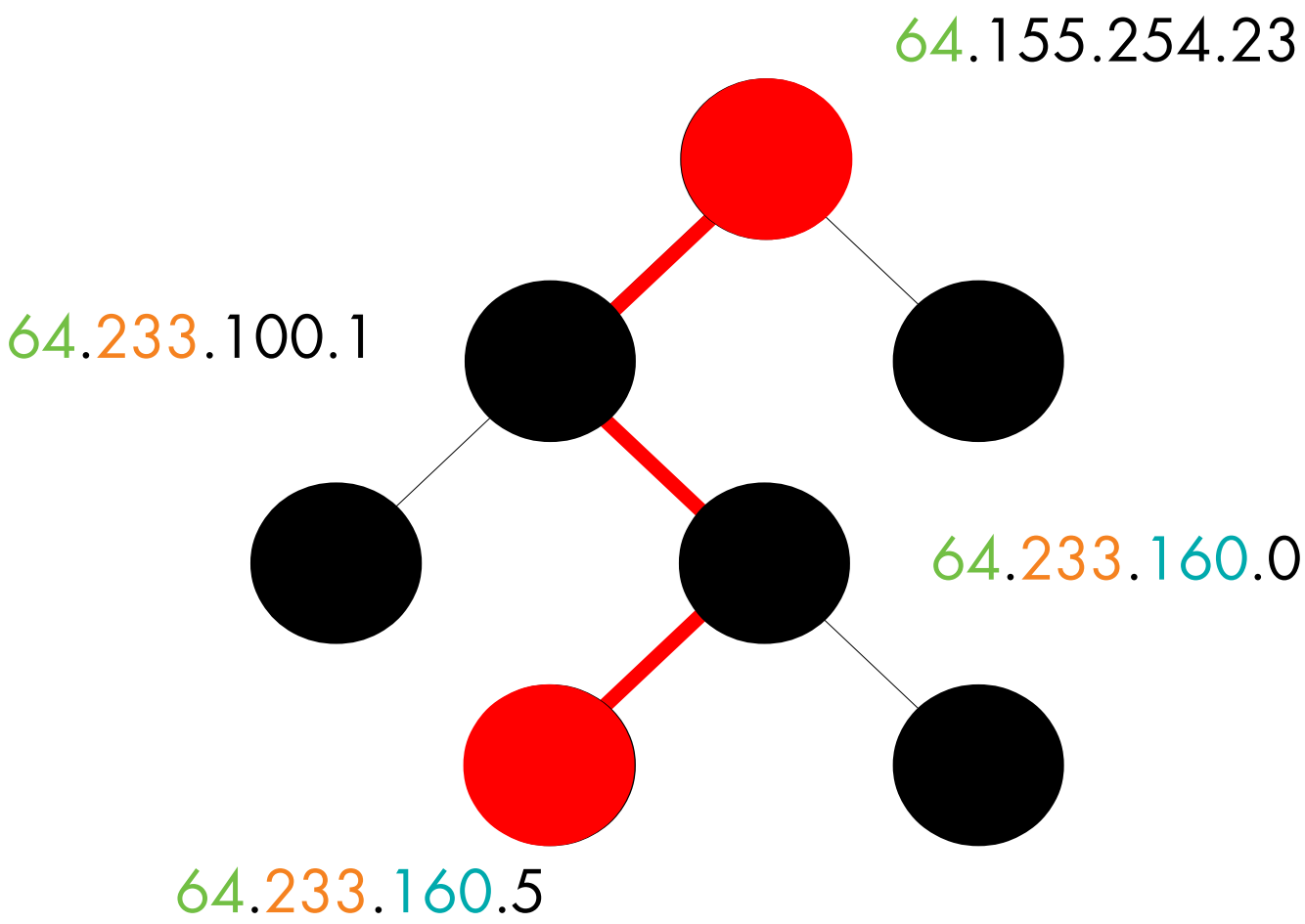
- 27 Bit in der Netzmaske gesetzt

	Dezimal	Binär	Berechnung
IP Adresse	130.094.122.195	10000010 01011110 01111010 11000011	*IP-Adresse*
Netzmaske	255.255.255.224	11111111 11111111 11111111 11100000	AND *Netzmaske*
Netzwerkadr.	130.094.122.192	10000010 01011110 01111010 11000000	= *Netzwerkteil*
IP Adresse	130.094.122.195	10000010 01011110 01111010 11000011	*IP-Adresse*
Netzmaske	255.255.255.224	11111111 11111111 11111111 11100000	
		00000000 00000000 00000000 00011111	AND(NOT *Netzmaske*)
Geräteteil	3	00000000 00000000 00000000 00000011	= *Geräteteil*

Routing von Datenpaketen in IP-Netzwerken

- Vergleich von Quell- und Ziel-IP-Adresse
- Übereinstimmung Netzwerkteile? → direkter Versand an Empfänger
- Unterschiedlicher Netzwerkteile? → Weiterleiten über Router
  - Ermittlung der IP-Adresse eines Routers (über Routingtabelle)
  - Senden des Datenpaketes an den Router
- Router verfährt nach dem selben Verfahren
- Beim Transport können bis zur Zustellung am Endgerät mehrere Netze und Router (Hop) durchlaufen werden

Identifikation der Knoten per IP-Adresse



Klassifizierung von Adressen

- Drei IP-Adress-Typen
  - Unicast: Senden an einen einzigen bestimmten Empfänger
  - Broadcast: Senden an alle Geräte im selben Netz (Subnetz)
  - Multicast: Senden an einige Geräte im selben Netz
- Lokales Netz (localhost, Loopback):127.0.0.1/8
- Privates Netzwerk:192.168.0.0/16
- Multicast-Adressen: 224.0.0.0/4
- Netzunabhängige Broadcastadresse: 255.255.255.255

Übersicht

- Grundlagen Interprozesskommunikation
- Internet Protokoll
  - Grundlagen
  - IPv4
  - IPv6
- Transmission Control Protocol
- User Datagram Protocol
- Sockets
- Realisierung in C++

„Mehr als 640 Kilobyte Speicher werden Sie niemals benötigen."
[https://www.express.de/5757638]

IPv6

- Nachfolger für IPv4
- Relativ geringe Anzahl verfügbarer IPv4-Adressen (4.294.967.296)
- Absehbare Verknappung im Zeitverlauf
- Daher IPv6 mit Verwendung 128 Bit großer Netzwerkadressen
  - Vergrößerung des Adressraumes auf 2^{128} Adressen
  - Keine Subnetzmasken, Angabe der Präfixlänge als Äquivalent
  - Hexadezimale Angabe von 8 Blöcken zu jeweils 16 Bit
- Bisher noch keine große Verbreitung (bei Endanwendern marginal)
- Begründet auch durch mögliche Koexistenz mit IPv4

IPv6 Aufbau und Beispiel

- Beispiel: Netzwerkgerät mit IPv6-Adresse
  - 2001:0db8:85a3:08d3:1319:8a2e:0370:7347/64,
- Präfix belegt die 64 linken Bits der IPv6-Adresse
  - Präfix = Bits der Netzwerkkennung
  - Im Beispiel lautet Präfix 2001:0db8:85a3:08d3
- Interface Identifier (die 64 rechten Bits der Adresse)
  - Eindeutige Identifizierung des Endknotens
  - Im Beispiel lautet Interface Identifier 1319:8a2e:0370:7347
  - IdR. aus eindeutiger MAC-Adresse der Netzwerkkarte erstellt
  - Auch variable Vergabe möglich
  - Media-Access-Control-Adresse(MAC) für Gerät konstant einmalig fest

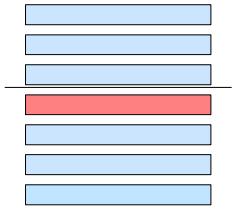
IPv6 Erweiterungen (Auswahl)

- Optimierung Paket-Header zur besseren Verarbeitung durch Router
- Autokonfiguration von IPv6-Adressen
- Mobile IP
  - Erlaubt Endgeräten überall unter gleicher IP-Adresse erreichbar zu sein
- IPSecurity (IPSec)
  - Verschlüsselung, Vertraulichkeit, Authentizität, Integrität
- Quality of Service (QoS)
  - Sicherstellung einer bestimmten Dienstgüte (Latenz, Jitter, Durchsatz)
- Wiederherstellung des Ende-zu-Ende Prinzips
  - Unter IPv4 aufgrund der Adressknappheit entsprechend umgangen

Übersicht

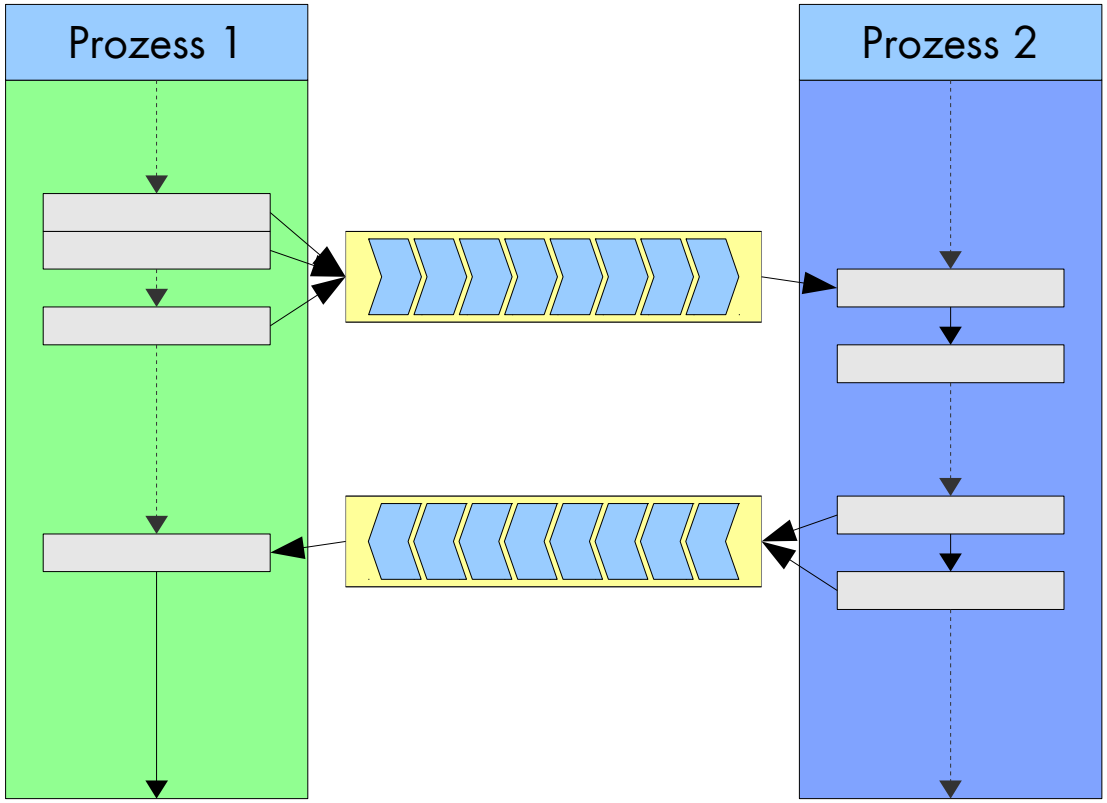
- Grundlagen Interprozesskommunikation
- Internet Protocoll
- Transmission Control Protocol
- User Datagram Protocol
- Sockets
- Realisierung in C++

Transmission Control Protocol (TCP)



- Protokoll der Transportschicht
- Erste Standardisierung 1981
- Verbindungsorientiertes Protokoll
  - Funktionen für Auf- und Abbau einer virtuellen Verbindung zwischen zwei eindeutig definierten Endpunkten
  - Bereitstellung eines bidirektionalen Kommunikationskanals
- Nutzende Protokolle (u.a.)
  - *Hypertext Transfer Protocol* (HTTP)
  - *Secure Shell* (SSH)
  - *(Secure) File Transfer Protocol* ((S)FTP)
  - *Internet Message Access Protocol* (IMAP)

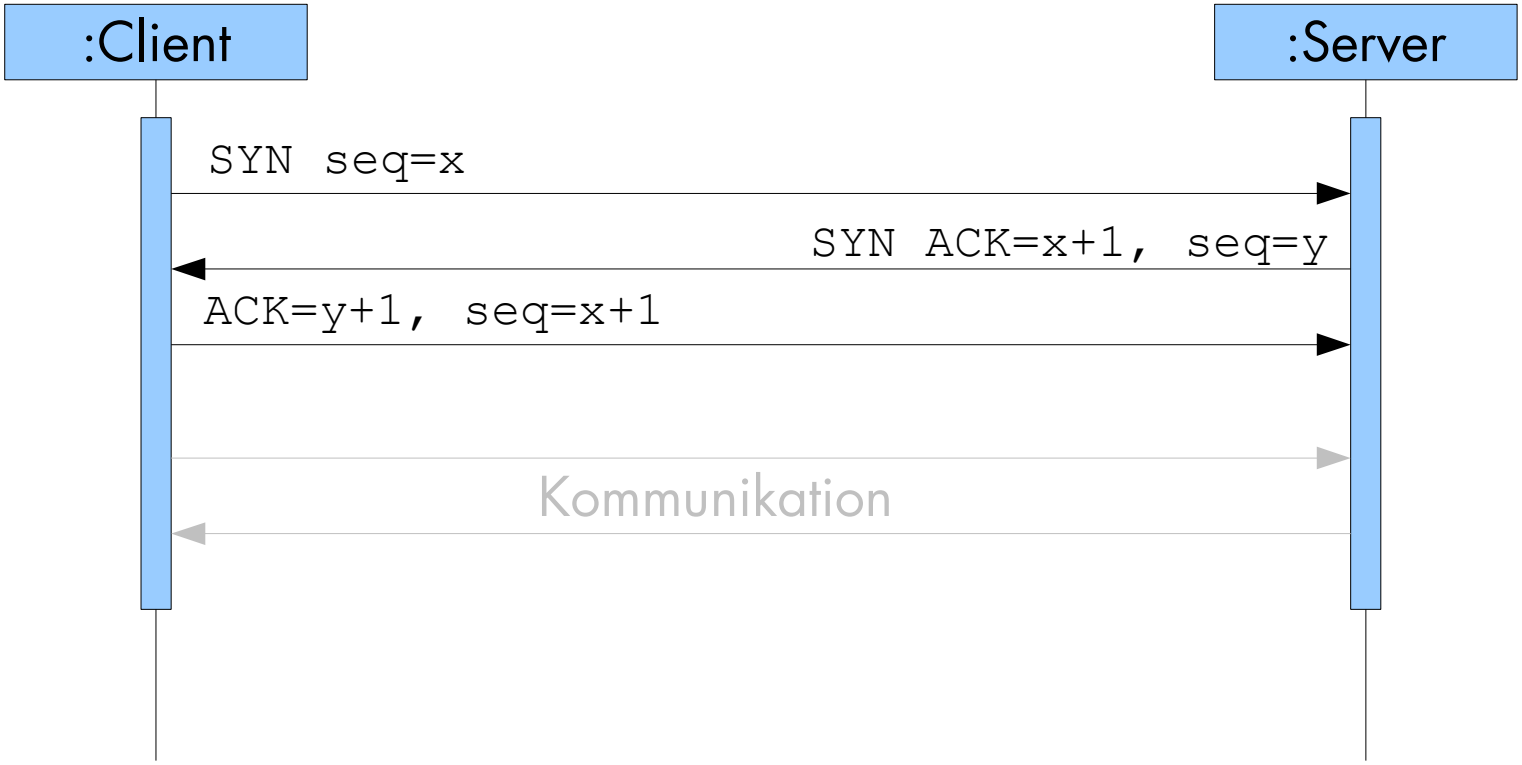
IPK Pipes



Vorteile von TCP

- Verlust von Daten wird erkannt und automatisch korrigiert
  - Pakete werden vom Sender laufend nummeriert
  - Empfänger bestätigt den Erhalt jedes Pakets
  - Bei Fehlen einer Bestätigung wird das fehlende Paket erneut verschickt
- Verifikation der Datenpakete über Prüfsumme
- Reihenfolgetreue
  - Daten nur in der korrekten Reihenfolge an aufrufende Schicht
  - Doppelte Pakete werden ausgesondert
- Erkennung von Überlast, wenn gesendete Pakete nicht bestätigt
  - Beispielsweise durch Pufferüberläufe an Routern oder Empfänger
  - Reduzierung der Übertragungslast (übertragene Pakete pro Zeiteinheit)
  - Bei erneutem Eingang von Bestätigungen wird die Übertragungslast wieder schrittweise erhöht

Aufbau einer TCP-Verbindung - 3 Wege Handschlag



Backup

Aufbau einer TCP-Verbindung - 3 Wege Handschlag

1)

- Client sendet Server spezielles *SYN-Paket* mit Sequenznummer x
- Sequenznummer ist eine beliebige, generierte Zahl
- Notwendig für Sicherstellung der korrekten Übertragungsreihenfolge

2)

- Server empfängt das Paket
- Port geschlossen, so wird ein *TCP-RST-Paket* gesendet
- Port offen, so sendet der Server ein *SYN-ACK-Paket* als Bestätigung
- Enthält auch die um eins erhöhte Sequenznummer x
- Zusätzlich eine beliebige Start-Sequenznummer y mitgesendet

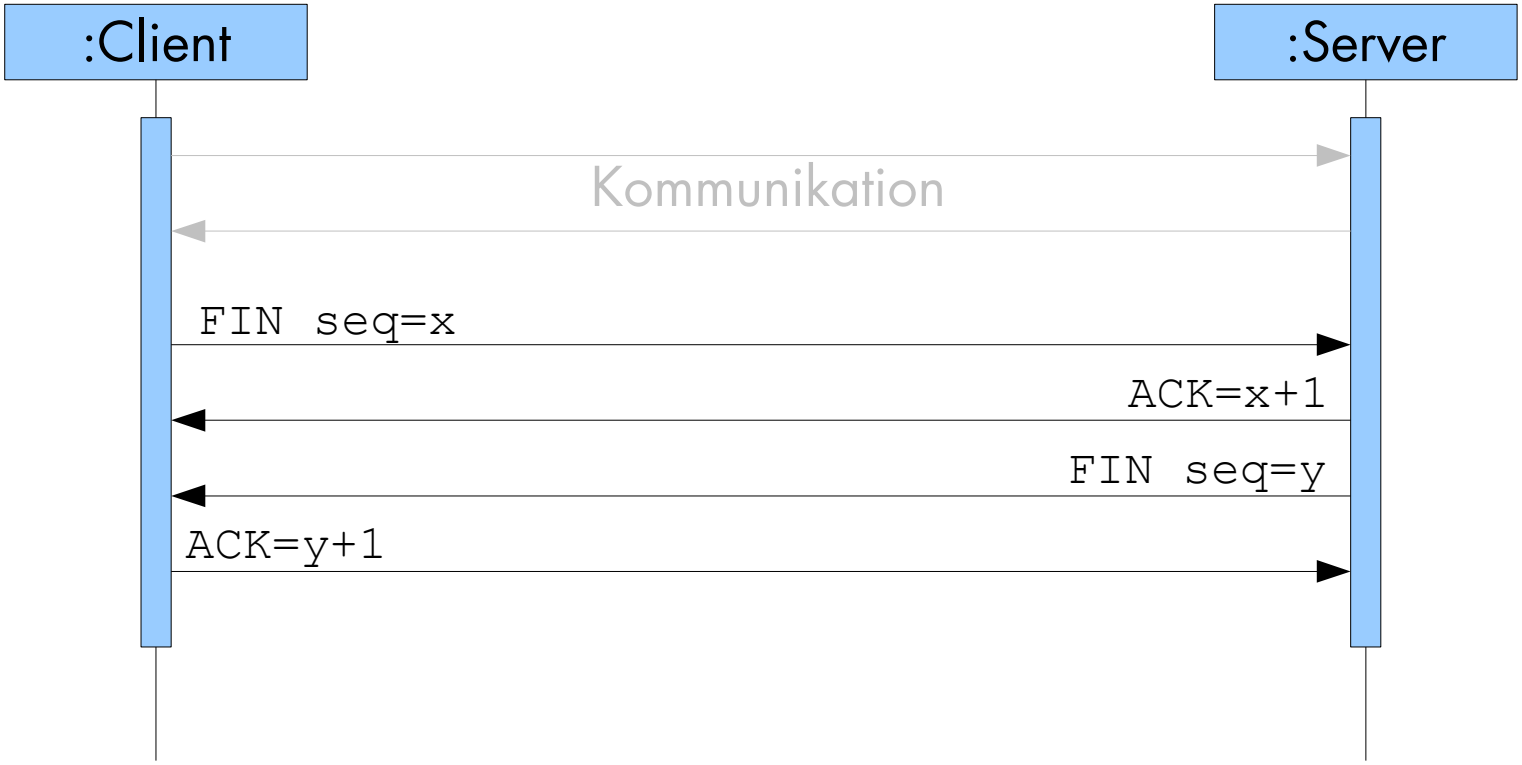
Backup

Aufbau einer TCP-Verbindung

3)

- Client bestätigt Erhalt des *SYN-ACK-Pakets* mit einem *ACK-Paket*
- Enthält jeweils um eins erhöhte Sequenznummern x und y
- Damit ist der Verbindungsaufbau abgeschlossen
- Ab jetzt gleichberechtigte Kommunikation für beide Seiten
- Grenze zwischen Client und Server verschwimmt

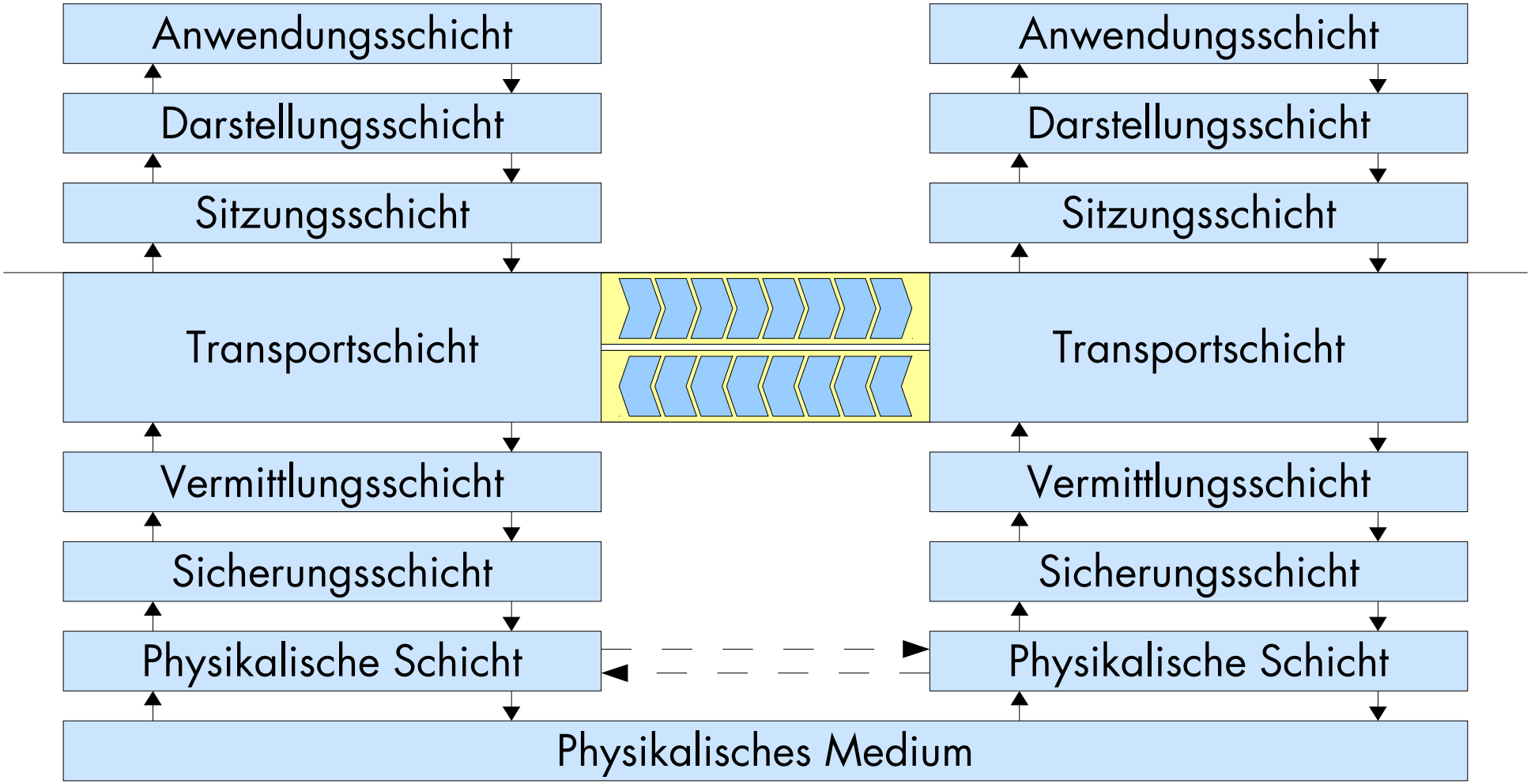
Abbau einer TCP-Verbindung



Abbau einer TCP-Verbindung

- Einer der beiden Kommunikationsteilnehmer (Client oder Server) sendet *FIN-Paket* und Sequenznummer x
- Bestätigung durch *ACK-Paket* mit Sequenznummer x+1
- Empfänger sendet ein *FIN-Paket* mit Sequenznummer y
- Empfänger sendet noch ein *ACK-Paket* mit Sequenznummer y+1
- Damit ist der Verbindungsabbau abgeschlossen

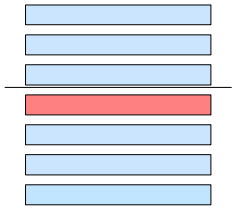
„Logische“ Kommunikation über TCP



Übersicht

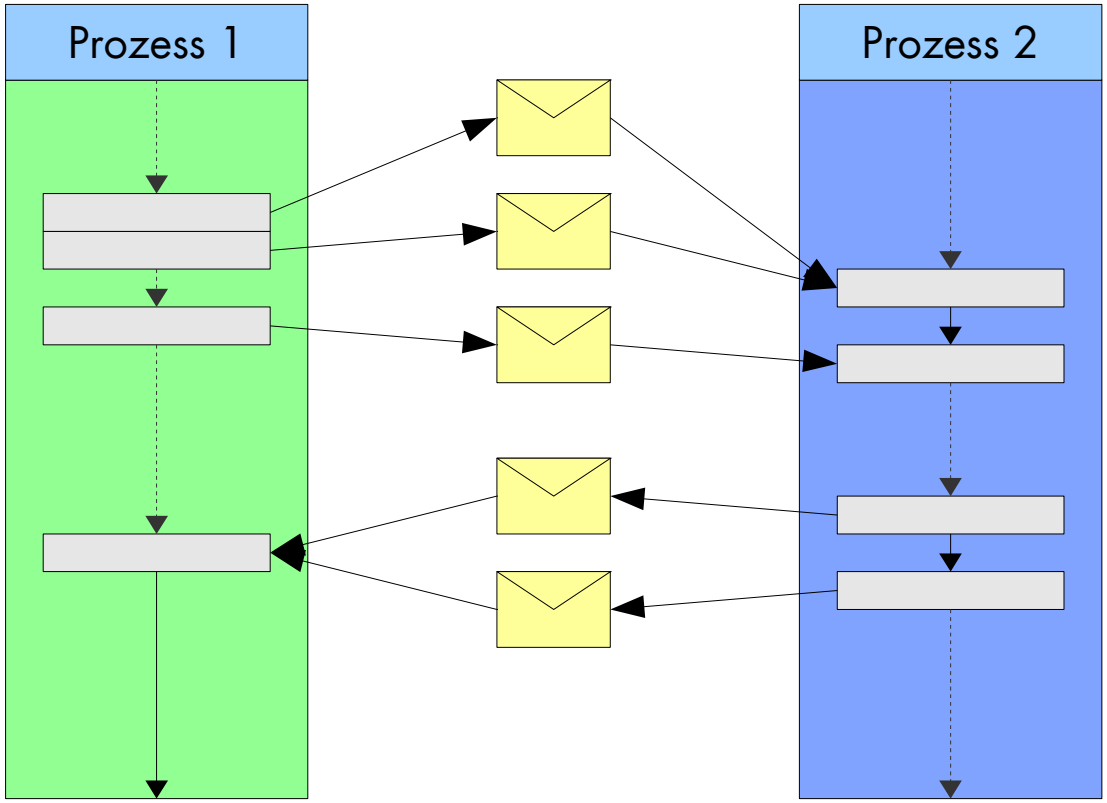
- Grundlagen Interprozesskommunikation
- Internet Protocoll
- Transmission Control Protocol
- User Datagram Protocol
- Sockets
- Realisierung in C++

User Datagram Protocol (UDP)



- Protokoll der Transportschicht
- Verbindungsloses Protokoll
  - Vor der Kommunikation muss keine Verbindung aufgebaut werden
  - Keine Garantie, dass Pakete ankommen
  - Keine Garantie, dass Pakete in der gesendeten Reihenfolge ankommen
  - Keine Garantie, dass Pakete nicht doppelt ankommen
- Verlusterkennung/-korrektur, Reihenfolgetreue und Überlastkontrolle müssen wenn benötigt auf anderer Schicht realisiert werden
- Verifikation der Integrität eines Pakets über optionale Prüfsumme

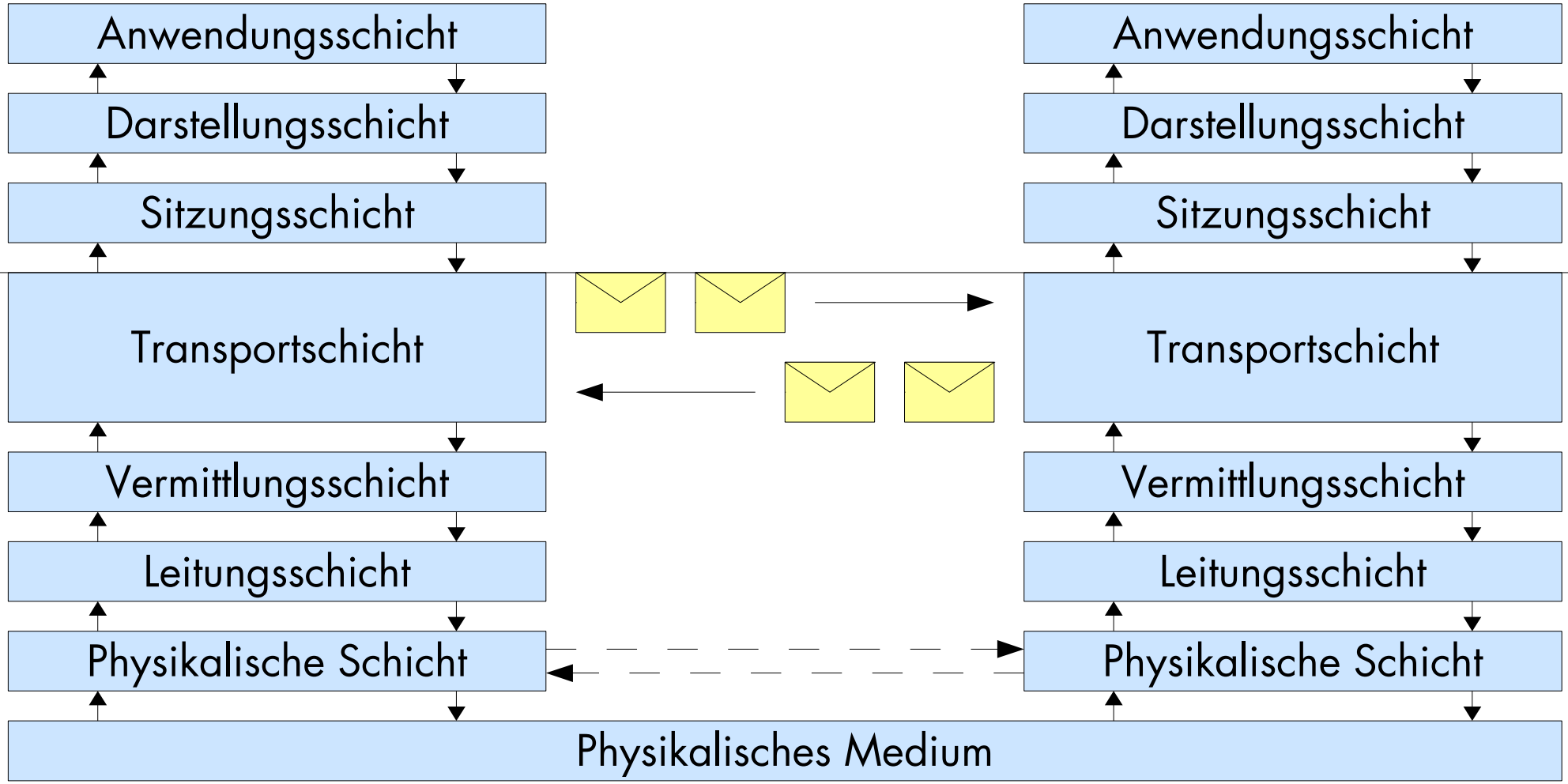
IPK Messages



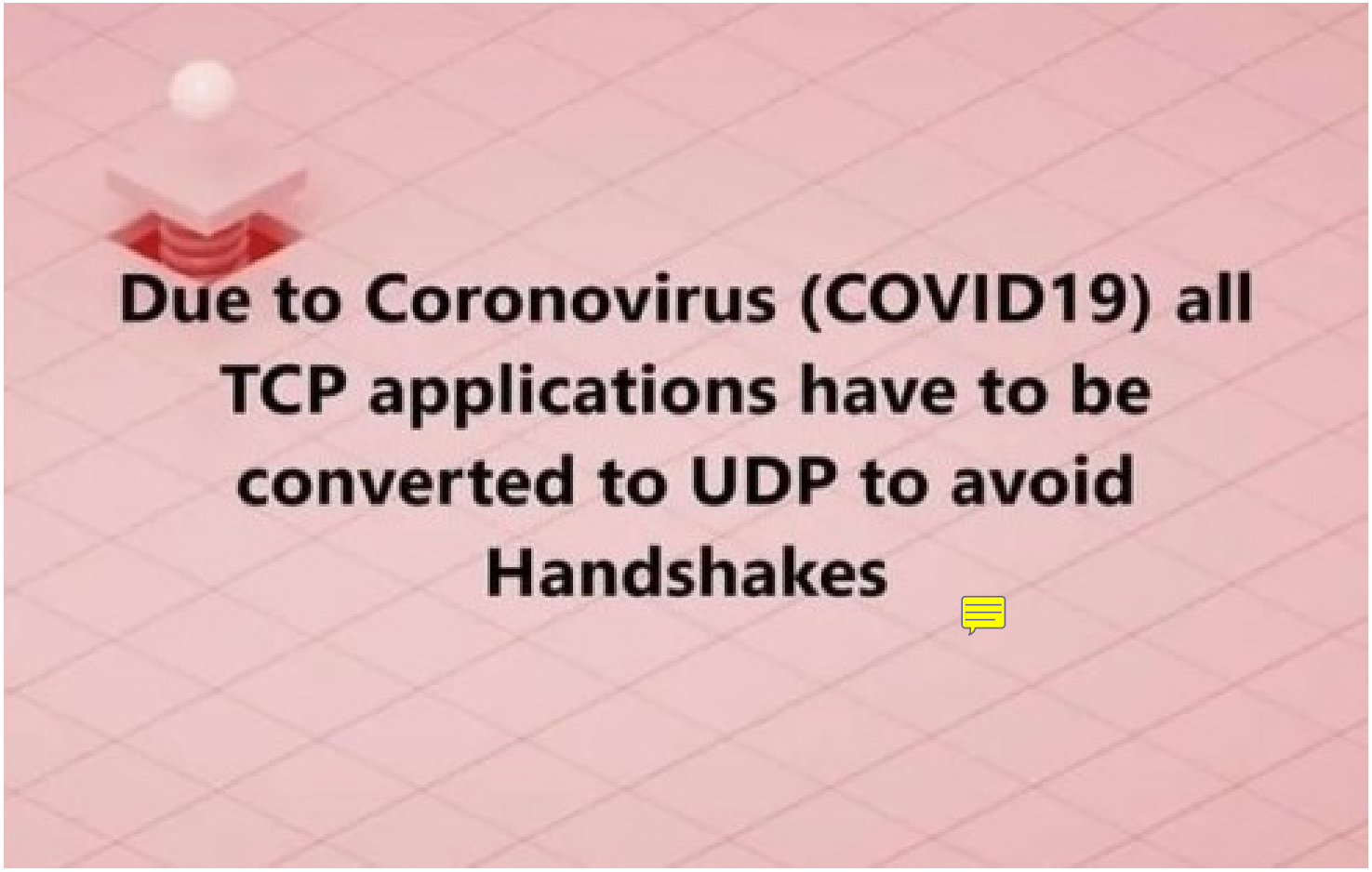
User Datagram Protocol (UDP)

- Sinnvoll bei kleinen Datenmengen
  - Overhead des 3-Wege-Handschlag zu groß
  - Fokus auf schnellen kurze Kommunikation
- Weniger Netzwerklast
  - Keine automatische Neusendung verloren gegangener Pakete
  - Keine Bestätigungspakete notwendig
- Nutzende Protokolle
  - *Domain Name System* (DNS)
  - *Dynamic Host Configuration Protocol* (DHCP)

„Logische“ Kommunikation über UDP



TCP in der Krise...



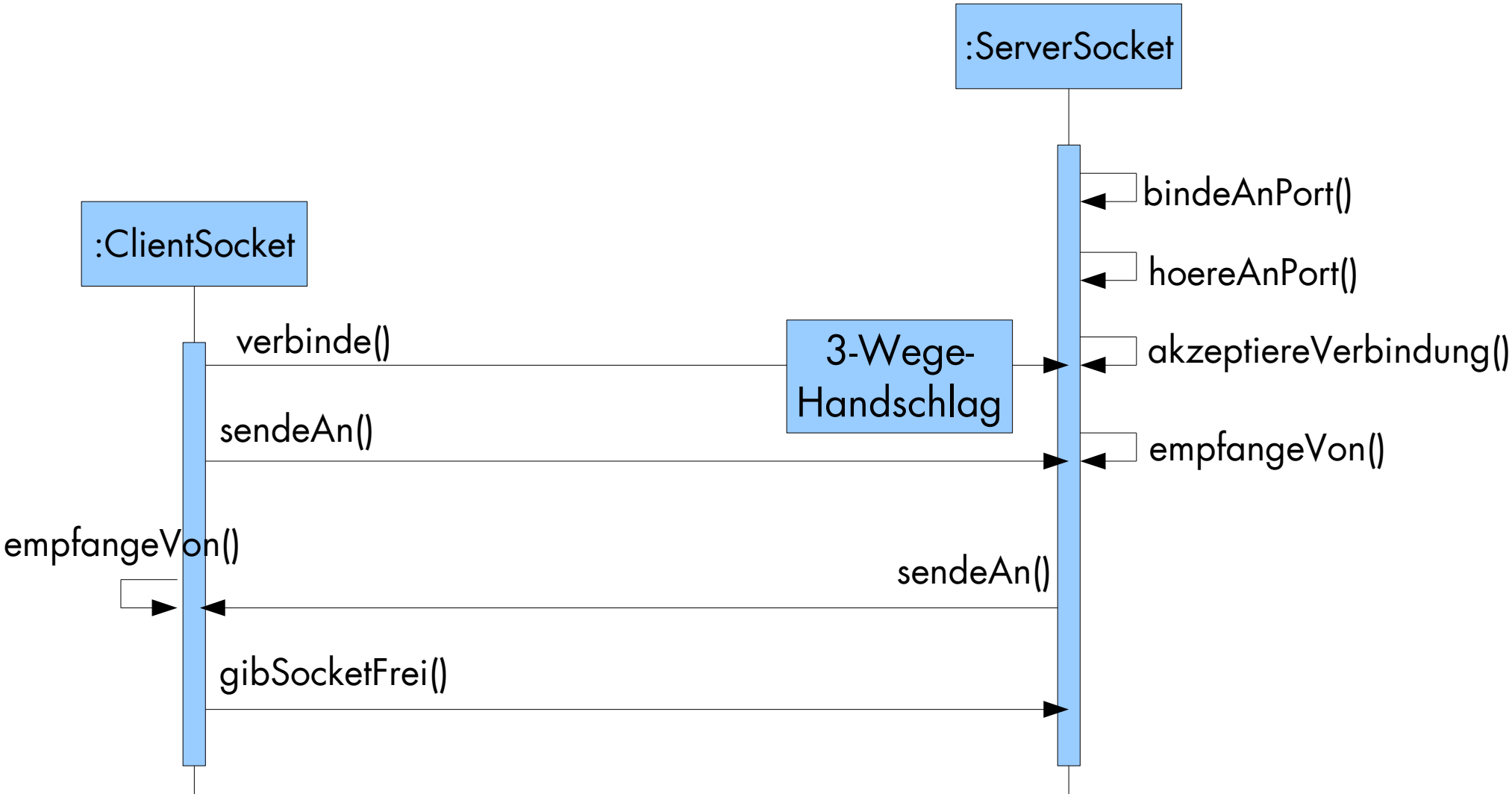
Übersicht

- Grundlagen Interprozesskommunikation
- Internet Protocoll
- Transmission Control Protocol
- User Datagram Protocol
- Sockets
- Realisierung in C++

Socket

- Bidirektionale Softwareschnittstelle zur Interprozesskommunikation
- Abstrakte Schnittstelle zwischen Netzwerk-Protokoll-Implementierung des Betriebssystems und der Anwendungssoftware
- Zur Kommunikation werden zwei Sockets benötigt
  - Client-seitiger Socket und Server-seitiger Socket
  - Adressiert durch jeweils einen eigenen Port
- Aufbauend auf der Transportschicht, abstrahieren sie von den zugrunde liegenden Netzwerkprotokollen

Verbindungsorientierte - Socket-Kommunikation



Abfolge Server-Socket

- Server-*Socket* erzeugen und an *Port* binden
- Server wartet auf Verbindung mit Client-*Socket* - „auf *Port* lauschen“
- Bei eingehender Verbindung mit Client-*Socket*
  - Verbindung akzeptieren
  - Ein- und Ausgabeströme der Kommunikationsverbindung öffnen
  - Kommunikation mit dem Client-*Socket* über die Ein- und Ausgabeströme ggf. nach einem spezifischen Protokoll
  - Schließen der Ein- und Ausgabeströme
  - Server-*Socket* schließen
  - Ggf. Rückkehr in die Warteschleife

Abfolge Client-Socket

- Client-*Socket* erzeugen und mit Server-*Socket* verbinden
- Ein- und Ausgabeströme der Kommunikationsverbindung öffnen
- Kommunikation mit dem Server-*Socket* über die Ein- und Ausgabeströme ggf. nach einem spezifischen Protokoll
- Schließen der Ein- und Ausgabeströme
- Client-*Socket* schließen

Problem der Heterogenität

- Heterogenität der Knoten nicht vollständig durch Sockets aufgelöst
- Muss auf anwendungsorientierten Schichten selbst gelöst werden
- Beispiel: Interne Darstellung von Zahlen und Zeichen
- Reihenfolge der Abspeicherung von Bytes der Mehr-Byte-Datentypen im Speicher
- Endian-Problem
  - Big-Endian-Systeme speichern höherwertige Bytes auf niedrigeren Speicheradressen
  - Little-Endian-Systeme speichern höherwertige Bytes auf höheren Speicheradressen

Endian-Problem

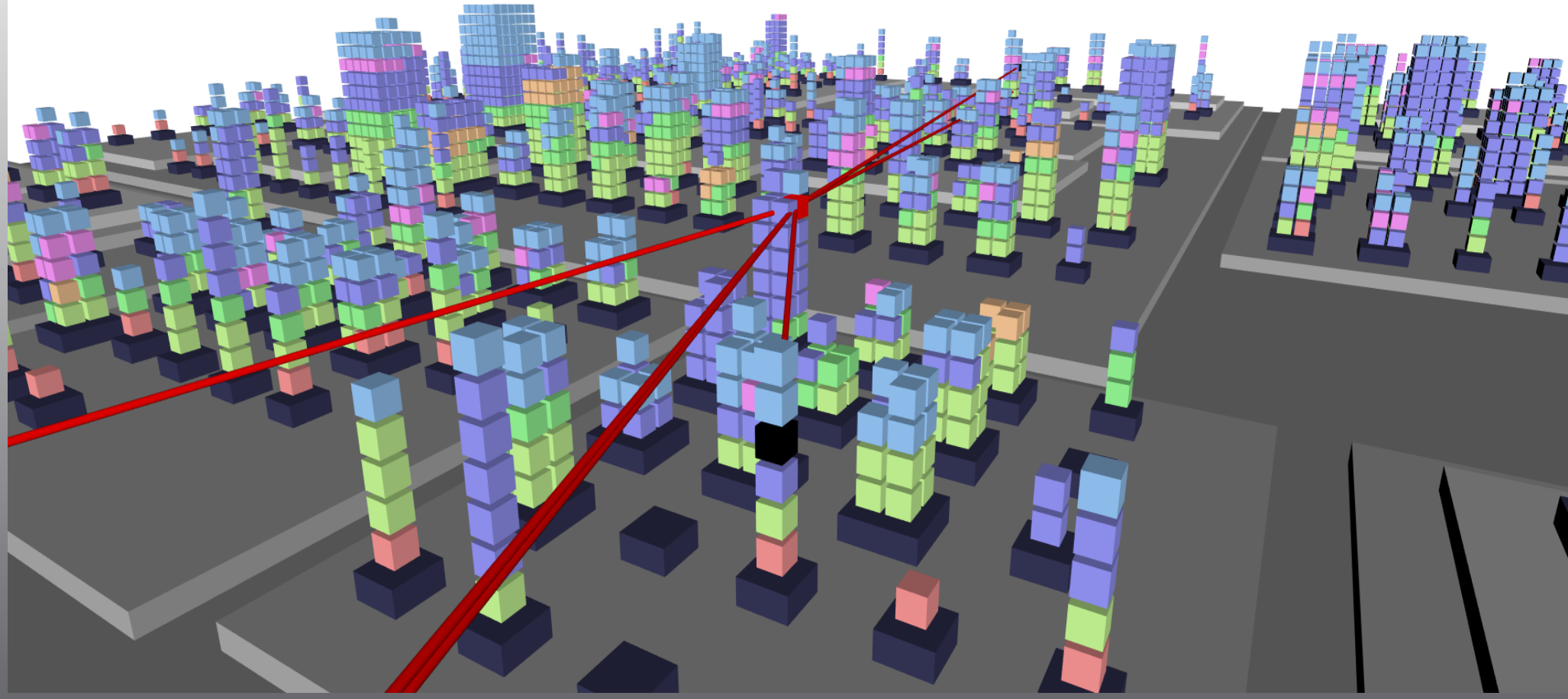
- Beispiel
  - 1.440 = 00000000 00000000 00000101 10100000
- Speicheradresse Big-Endian-Darstellung Little-Endian-Darstellung
- 00 00000000 10100000
- 01 00000000 00000101
- 02 00000101 00000000
- 03 10100000 00000000
- Übertragung in vereinbarter Network Byte Order (TCP/IP: Big-Endian-Format → Sockets)
- BigEndian 1.440
- LittleEndian 10.738.728.960

Übersicht

- Grundlagen Interprozesskommunikation
- Internet Protocoll
- Transmission Control Protocol
- User Datagram Protocol
- Sockets
- Realisierung in C++

Sockets in C/C++

- Quasi-Standard: Berkeley-Sockets
  - Für die meisten modernen UNIX-Systeme (z. B. Linux, BSD) und MS Windows ab 9x und NT
  - Implementierung in C
- Probleme bei Plattformunabhängigkeit
  - Funktionsdefinitionen in verschiedenen Header-Dateien
  - Unter MS Windows: winsock.h bzw. winsock2.h

SimpleTCPSocket

Vielen Dank für Ihre Aufmerksamkeit